

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 11

Course: Operating Systems

Topic: The Race and the Semaphore

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 11 · The Race and the Semaphore

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 11 (Track A: The Race and the Semaphore, Linux)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.3.1-1-cachyos)
- **Compiler:** GCC 16.2.1 20260810 (`gcc -Wall -pthread`)
- **Threading Library:** POSIX Threads (`libpthread` / glibc 2.40)
- **Shell:** GNU bash, version 5.3.20(1)-release
- **Workspace:** `/home/ilie/Documents/SO/os-lab11` (`~/os-lab11`)

Purpose of the Work

1. Investigate the emergence of race conditions in multi-threaded programs accessing shared memory without synchronization primitives.
2. Disassemble and analyze the non-atomic nature of arithmetic increment expressions (`counter++`) at the CPU instruction set level.
3. Eliminate concurrency anomalies using POSIX mutual exclusion locks (`pthread_mutex_t`), identifying critical sections and enforcing mutual exclusion.
4. Re-implement mutual exclusion using POSIX counting semaphores (`sem_t`) initialized to binary capacity (1), examining semaphore invariants and counter degradation under higher initialization values.
5. Benchmark and profile execution overhead across uncoordinated execution, mutex locks, semaphore operations, and lock-free hardware atomics (`stdatomic.h`).
6. Apply semaphores as unidirectional ordering signals initialized to 0 to synchronize asynchronous producer-consumer relationships between threads.
7. Implement resource pooling and multi-unit throttling using counting semaphores in a concurrent parking lot simulation, evaluating boundary conditions when capacity limits are modified.

Part 1. See the Race

1. Architectural Concept

In shared-memory symmetric multiprocessing (SMP), threads share the same virtual address space, heap, and global variables while executing with independent CPU register states and thread-local stacks. When multiple threads read and modify a shared variable concurrently without synchronization, the outcome depends on the unpredictable cycle-level interleaving of machine instructions, producing a **race condition**.

Although the high-level C statement `counter++` appears atomic, the compiler decomposes it into three distinct machine instructions on load-store architectures: 1. **Read (Load):** Transfer data from memory into a CPU register. 2. **Modify (Add):** Increment the value in the register. 3. **Write (Store):** Write the updated register value back to main memory.

When two threads execute these three stages concurrently, an intervening context switch or parallel hardware execution causes one thread to overwrite the other's modification with stale data, resulting in lost updates.

2. Implementation: `race.c`

The program launches two POSIX threads, each tasked with executing an increment loop $1,000,000$ times on the global variable `counter`:

```c
#include <stdio.h>
#include <pthread.h>

#define N 1000000
long counter = 0; /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++; /* read, add 1, write back: three steps */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

Figure 1.1 – Implementation of uncoordinated concurrent counter increment in race.c

3. Compilation and Execution

We compile without `-O2` optimization to prevent the compiler from lifting `counter` into registers across the entire loop:

```
$ gcc -Wall -pthread -o race race.c
$ ./race
counter = 1013917 (expected 2000000)
$ ./race
counter = 1337324 (expected 2000000)
$ ./race
counter = 1255795 (expected 2000000)
```

Figure 1.2 – Execution output of race demonstrating severe lost update anomalies

Inspecting the assembly code generated by GCC for the inner loop verifies the instruction decomposition:

```
.L3:
    movq    counter(%rip), %rax    # Step 1: Read memory into register %rax
    addq    $1, %rax               # Step 2: Add 1 to register %rax
    movq    %rax, counter(%rip)    # Step 3: Write back %rax into memory
    addl    $1, -4(%rbp)           # Loop index increment
.L2:
    cmpl    $999999, -4(%rbp)
    jle     .L3
```

Figure 1.3 – Disassembled x86-64 assembly instructions for counter++

Observe (Part 1 Answers)

1. **counter++ looks like one step. Write the three steps the CPU really does:**
   - **Step 1 (Load):** `movq counter(%rip), %rax` reads the value of `counter` from RAM/L1-cache into general-purpose register `%rax`.
   - **Step 2 (Modify):** `addq $1, %rax` increments the register `%rax` by 1 in the Arithmetic Logic Unit (ALU).
   - **Step 3 (Store):** `movq %rax, counter(%rip)` commits the updated value from `%rax` back to memory.
2. **Describe, with those steps, how two threads can lose one addition:** Assume `counter` currently holds value 100:
   - Thread A executes `movq counter(%rip), %rax` (Thread A `%rax` = 100).
   - Thread B executes `movq counter(%rip), %rax` concurrently (Thread B `%rax` = 100).

- Thread B executes `addq $1, %rax` (Thread B `%rax` = 101) and `movq %rax, counter(%rip)` (memory now holds 101).
- Thread A resumes, executes `addq $1, %rax` (Thread A `%rax` = 101), and executes `movq %rax, counter(%rip)` (overwriting memory with 101). Although two independent addition operations occurred, the shared counter increased by only 1, resulting in a permanently lost addition.

3. **Why is the result different at every run?** Execution interleaving between threads is governed non-deterministically by OS kernel preemptive scheduling, hardware timer interrupts, CPU core frequency variations, and bus contention in the multi-core cache coherence subsystem (MESI protocol). Because threads overlap at different instruction boundaries during each run, the quantity of lost updates fluctuates randomly.

Part 2. Fix It: Mutex, then Semaphore

1. Mutex Protection (`mutex.c`)

A mutex (**mutual exclusion**) primitive guarantees that at most one thread can execute within a critical section at any given time. If a thread attempts to acquire an already-locked mutex via `pthread_mutex_lock`, it transitions to an OS-managed sleep state until the owning thread releases the lock via `pthread_mutex_unlock`.

```c
#include <stdio.h>
#include <pthread.h>

#define N 1000000
long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);   /* Enter critical section */
        counter++;
        pthread_mutex_unlock(&lock); /* Exit critical section */
    }
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

Figure 2.1 – Implementation of mutex-synchronized counter increment in mutex.c

2. Binary Semaphore Protection (`sem.c`)

A Dijkstra semaphore is an integer counter with two fundamental atomic operations: - `sem_wait(&s)` (P-operation / down): decrements the semaphore counter. If the value is ≤ 0, the calling thread blocks until another thread increments it. - `sem_post(&s)` (V-operation / up): increments the semaphore counter and wakes a waiting thread if any exist.

Initializing a POSIX semaphore with `sem_init(&s, 0, 1)` restricts its state space between 0 and 1, causing it to function identically to a mutual exclusion lock.

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 1000000
```

```c
long counter = 0;
sem_t s;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);   /* Decrement counter; wait if 0 */
        counter++;
        sem_post(&s);   /* Increment counter and wake waiter */
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1); /* Initial capacity: 1 */
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    sem_destroy(&s);
    return 0;
}
```

Figure 2.2 – Implementation of binary semaphore synchronization in sem.c

3. Execution and Performance Benchmarking

Both synchronization primitives produce the exact expected value of $2,000,000$ reliably:

```
$ gcc -Wall -pthread -o mutex mutex.c
$ ./mutex
counter = 2000000 (expected 2000000)
$ gcc -Wall -pthread -o sem sem.c
$ ./sem
counter = 2000000 (expected 2000000)
```

Figure 2.3 – Verified deterministic execution outputs of mutex and sem

We measure the execution wall-clock time across the three programs using the `time` utility:

```
$ time ./race
counter = 1159018 (expected 2000000)
real    0m0.008s
user    0m0.014s
sys     0m0.001s

$ time ./mutex
counter = 2000000 (expected 2000000)
real    0m0.083s
user    0m0.086s
sys     0m0.076s

$ time ./sem
counter = 2000000 (expected 2000000)
real    0m0.084s
user    0m0.061s
sys     0m0.089s
```

Figure 2.4 – Performance benchmark comparing race, mutex, and semaphore execution

Observe (Part 2 Answers)

1. **Run mutex and sem three times each. Is the answer always 2 000 000?** Yes. On every run, both programs produce exactly `counter = 2000000`. By constraining the concurrency invariant so that strictly one thread can occupy the critical section, the three-instruction sequence `movq` → `addq` → `movq` executes with complete atomicity relative to the other thread.

2. **What would happen in sem.c with sem_init(&s, 0, 2)? Try it and explain:** Initializing the semaphore with capacity 2 allows up to two threads to enter the critical section simultaneously without blocking. Because our application consists of exactly two threads, `sem_wait(&s)` immediately decrements the counter from 2 → 1 for Thread A and from 1 → 0 for Thread B. Consequently, both threads execute `counter++` concurrently, completely nullifying mutual exclusion. Running an experiment with `sem_init(&s, 0, 2)` confirmed lost updates:

   ```
   counter = 1987295 (expected 2000000)
   ```

3. **Why is the locked version slower?** The locked version is roughly 10× slower than the unprotected race version (0.083 s vs. 0.008 s) due to four primary architectural costs:

   - **Atomic Hardware Operations:** Mutex acquisition uses atomic test-and-set instructions (`lock cmpxchg` on x86) which serialize the CPU instruction pipeline and force memory bus arbitration.
   - **Cache Invalidation & Bus Traffic:** Modifying the shared lock structure forces cache lines to bounce between CPU core caches via the cache coherence protocol.
   - **Kernel Transitions (Futex):** When lock contention occurs, glibc invokes the Linux `sys_futex` system call, forcing expensive user-kernel privilege mode transitions and context switches.
   - **Serialization Loss:** The execution of both threads is completely serialized, forfeiting parallel multi-core CPU execution.

Part 3. A Semaphore as a Signal

1. Conceptual Mechanism

Beyond mutual exclusion, semaphores serve as asynchronous signaling primitives to enforce **execution ordering**. To guarantee that Thread B executes after Thread A completes an action, a semaphore is initialized to 0: - Thread B calls `sem_wait(&ready)` and immediately blocks because the semaphore value is 0. - Thread A performs its work, writes shared data, and subsequently calls `sem_post(&ready)` (0 → 1). - The kernel unblocks Thread B, allowing it to safely access the shared data.

2. Implementation: `order.c`

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready; /* starts at 0: nothing is ready yet */

void *writer(void *arg) {
    sleep(1); /* pretend the work takes a while */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready); /* 0 -> 1: wake up B */
    return NULL;
}

void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready); /* blocks while the value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
```

```c
    pthread_create(&b, NULL, reader, NULL); /* B starts first, on purpose */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

Figure 3.1 – Implementation of order synchronization using a signaling semaphore

3. Execution Verification

```
$ gcc -Wall -pthread -o order order.c
$ ./order
B: waiting for the message...
A: message written
B: I read "hello"
```

Figure 3.2 – Execution trace of order showing reader blocking until writer completion

Observe (Part 3 Answers)

1. **Change the start value to 1 (sem_init(&ready, 0, 1)), compile, run. What does B read now, and why?** When `ready` is initialized to 1, the semaphore acts as an already-granted permission ticket. Thread B calls `sem_wait(&ready)`, immediately decrements the counter from $1 \rightarrow 0$, and continues execution without blocking. Because Thread A sleeps for 1 second before populating `message`, Thread B reads uninitialized memory (an empty string `""`):

   ```
   B: waiting for the message...
   B: I read ""
   A: message written
   ```

2. **Put it back to 0. In one sentence: what does a semaphore that starts at 0 mean?** A semaphore initialized to 0 represents an unfulfilled dependency or closed gate, forcing any caller of `sem_wait` to sleep until another thread signals completion of an event via `sem_post`.

Part 4. A Semaphore as a Counter: The Parking Lot

1. Conceptual Mechanism

A counting semaphore initialized to value $K > 1$ represents a finite pool of K identical, interchangeable resources. Threads acquire a resource via `sem_wait` (decrementing capacity) and relinquish it via `sem_post` (incrementing capacity). When capacity reaches 0, subsequent threads block until a resource is returned.

2. Implementation: `parking.c`

In this simulation, 6 car threads compete for 3 parking spots (`PLACES = 3`). A mutex is included solely to ensure that the tracking variable `inside` and the corresponding terminal print statement remain synchronized.

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define PLACES 3
#define CARS   6

sem_t places; /* free places */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0; /* only for printing */

void *car(void *arg) {
    long id = (long)arg;
```

```c
    sem_wait(&places); /* take a place, or wait */
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sleep(1); /* parked */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places); /* give the place back */
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```

Figure 4.1 – Implementation of resource pool counting in parking.c

3. Execution Verification

```
$ gcc -Wall -pthread -o parking parking.c
$ ./parking
car 1 enters (inside: 1)
car 2 enters (inside: 2)
car 3 enters (inside: 3)
car 2 leaves (inside: 2)
car 1 leaves (inside: 1)
car 3 leaves (inside: 0)
car 4 enters (inside: 1)
car 6 enters (inside: 2)
car 5 enters (inside: 3)
car 6 leaves (inside: 2)
car 4 leaves (inside: 1)
car 5 leaves (inside: 0)
```

Figure 4.2 – Execution trace of parking.c demonstrating a maximum capacity of 3 concurrent cars

Observe (Part 4 Answers)

1. **What is the largest value of "inside" you see? Which line of code guarantees it?** The maximum observed value of `inside` is `3`. This upper bound is strictly guaranteed by the semaphore initialization `sem_init(&places, 0, PLACES)` where `PLACES = 3` in combination with `sem_wait(&places)` at the entrance of `car()`, which forces calling threads to sleep as soon as the available permit counter reaches 0.
2. **Set PLACES to 1. What does the semaphore become?** When `PLACES` is set to 1, the counting semaphore degrades into a binary semaphore (mutex), serializing all car arrivals such that strictly one car can occupy the lot at any given moment (`inside` alternates between 0 and 1).

Optional: Lock-Free Atomic Operations

For single-variable synchronization, the C11 standard (`stdatomic.h`) provides lock-free hardware atomics. The CPU instruction set guarantees that `atomic_fetch_add` performs the load, add, and store as an indivisible atomic bus cycle using the `lock` instruction prefix on x86_64, eliminating the need for OS kernel locks or sleep queues.

```c
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

#define N 1000000
atomic_long counter = 0;

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        atomic_fetch_add(&counter, 1);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", (long)counter, 2 * N);
    return 0;
}
```

Figure 5.1 – Lock-free atomic increment implementation in atomic.c

Executing `atomic` benchmarks at `0.022s`, nearly 4× faster than mutex/semaphore implementations while guaranteeing 100% precision without deadlocks or OS context switches:

```
$ gcc -Wall -pthread -o atomic atomic.c
$ time ./atomic
counter = 2000000 (expected 2000000)
real    0m0.022s
user    0m0.042s
sys     0m0.001s
```

Figure 5.2 – High-performance execution trace of lock-free atomic increment

---

Conclusions

In this laboratory work, we investigated concurrency hazards and synchronization primitives within shared-memory Linux environments: 1. **Root Cause of Race Conditions:** We demonstrated that high-level C statements such as `counter++` are decomposed by compilers into multiple assembly instructions (`movq` → `addq` → `movq`). Unsynchronized parallel execution across CPU cores causes interleaved instruction sequences that destroy data consistency. 2. **Mutual Exclusion:** By wrapping critical sections with POSIX mutex locks (`pthread_mutex_t`) or binary semaphores (`sem_t` initialized to 1), we eliminated race conditions, ensuring deterministic computation at the expense of synchronization and context-switch latency. 3. **Multi-Role Nature of Semaphores:** We established that semaphores provide three distinct synchronization patterns: - **Lock (Mutex):** Initialized to 1, guaranteeing exclusive single-thread critical section access. - **Signal:** Initialized to 0, coordinating unidirectional temporal dependencies between producer and consumer threads. - **Resource Counter:** Initialized to $K > 1$, enforcing bounded multi-unit concurrency limits across pooled resources. 4. **Hardware Atomics:** We contrasted OS-managed synchronization primitives with hardware-level atomic instructions (`stdatomic.h`), observing that bus-locked CPU primitives achieve correct results with significantly reduced latency.